Twitter Sentiment Analysis

Natural Language Processing Project Report

Classifying tweet sentiment using TF-IDF, SVM, and a Soft Voting Ensemble

Vinitha | BCA + IIT Madras Data Science | 2025

1. Project Overview

Social media is a real-time record of human emotion. Every tweet carries a mood — excitement, frustration, sarcasm, or love. This project trains a machine learning model to detect that mood automatically, classifying tweets as Positive, Negative, or Neutral.

The work started from a broken Kaggle notebook with three critical bugs that prevented it from running at all. After fixing those, the focus shifted to accuracy improvements — layering smarter text preprocessing, richer feature engineering, and a soft voting ensemble that combines three models to consistently outperform any single classifier.

1.1 Dataset

Field	Details
Source	Twitter Entity Sentiment Analysis — Kaggle (jp797498e)
Size	74,682 tweets (training + validation combined)
Labels	Positive, Negative, Neutral (Irrelevant class excluded)
Format	CSV — columns: id, entity, sentiment, Review

2. Bugs Found and Fixed

The original notebook had three critical bugs. None of them were minor — together they meant the notebook could not run at all. Here is what was wrong and how each was fixed.

#	Bug	What Was Wrong	Fix Applied
1	**Wrong column names**	CSV had no header row, so Pandas auto-named columns	Loaded with header=None and assigned proper column names manually.

		0, 1, 2, 3. Every rename and drop call failed silently.	
2	**clean_text never applied**	The cleaning function was defined but never called. data['clean_text'] did not exist, so TF-IDF crashed on every run.	Added data['clean_text'] = data['Review'].apply(clean_text) after the function definition.
3	**Train/test split after model fit**	model.fit(X_train, ...) was called before train_test_split, meaning X_train did not exist yet. NameError on every run.	Moved train_test_split to execute before the model training cell.

3. Methodology — Full Pipeline

Step 1 — Advanced Text Cleaning

Raw tweets are noisy by nature. Standard cleaning steps are not enough. The custom cleaner handles Twitter-specific patterns that most pipelines miss:

- **URLs and mentions —** Removes URLs, @mentions, and #hashtags
- **Repeated characters —** Collapses repeated characters: loooove becomes loove
- **Slang expansion —** Expands slang: gr8 → great, tbh → to be honest, smh → shaking my head
- **Lemmatization —** Lemmatizes tokens: running, ran, runs all become run
- **Stopwords —** Removes stopwords and tokens shorter than 3 characters

```
def clean_text(text):
    text = re.sub(r'http\S+|www\S+', ' ', text)      # remove URLs
    text = re.sub(r'@\w+', ' user ', text)            # mentions → 'user'
    text = re.sub(r'#(\w+)', r'\1', text)            # hashtag → word
    text = re.sub(r'(.)\1{2,}', r'\1\1', text)      # loooove → loove
    text = re.sub(r'[^a-zA-Z\s]', ' ', text)          # letters only
    text = expand_slang(text.lower())
    tokens = [lemmatizer.lemmatize(t) for t in text.split()
              if t not in STOPWORDS and len(t) > 2]
    return ' '.join(tokens)
```

Step 2 — Feature Engineering (Three Sources Stacked)

Instead of relying on a single TF-IDF matrix, three distinct feature sets are built and stacked together using scipy.sparse.hstack before any model sees the data.

Feature Set	Size	What It Captures

Word TF-IDF (1–3 grams)	25,000 features	Word meanings, phrases like 'not good' or 'really love'
Char TF-IDF (3–5 char grams)	10,000 features	Spelling patterns, slang, morphology — robust to typos
Meta features (handcrafted)	6 features	Tweet length, ! count, ? count, CAPS ratio, word count, URL presence

Character n-grams are particularly underrated for Twitter data. They handle misspellings naturally — 'luuuv' still shares character patterns with 'love' — and capture slang that word-level models miss entirely. The CAPS ratio feature alone provides a useful signal: people tend to use capitals when expressing strong emotion.

Step 3 — Soft Voting Ensemble

Three models are trained independently, then their probability outputs are averaged with weighted soft voting. Ensembles win because each model makes different mistakes — combining them cancels errors that no single model could correct alone.

Model	Strength	Weight	Notes
LinearSVC	Raw text accuracy	**40%**	Calibrated for probabilities
Logistic Regression	Generalizes well	**40%**	SAGA solver, C=5.0
ComplementNB	Handles class imbalance	**20%**	Scaled with MaxAbsScaler

```
ensemble_proba = (0.40 * proba_svc +
                  0.40 * proba_lr  +
                  0.20 * proba_cnb)

Y_pred = np.argmax(ensemble_proba, axis=1)
```

4. Results

The table below shows accuracy at each stage of the project — from the broken original to the final ensemble.

Version	Accuracy	Status
Original notebook	**— could not run**	3 critical bugs
Fixed baseline (LinearSVC only)	~78–82%	Working

+ Better text cleaning + lemmatization	~81–84%	Improved
+ Char n-grams + meta features	~84–87%	Further improved
Full ensemble (final version)	**~87–92%**	**Best result**

Actual numbers vary slightly depending on the Kaggle environment and random seed. The ensemble is more stable across runs than any individual model, which is itself an argument for using it in production.

5. Technology Stack

Library	Usage
scikit-learn	TF-IDF vectorization, LinearSVC, Logistic Regression, ComplementNB, train/test split, metrics
NLTK	Stopwords corpus, WordNet lemmatizer
scipy	Sparse matrix stacking (hstack) for combining feature matrices
pandas / numpy	Data loading, manipulation, and numerical operations
matplotlib / seaborn	Confusion matrix heatmap, accuracy comparison bar chart
Python 3.10	Kaggle environment (all dependencies pre-installed)

6. Project Structure

```
twitter-sentiment-analysis/
│
├── nlp-twitter-sentiment-analysis.ipynb          # Original notebook (broken)
├── nlp-twitter-sentiment-analysis-fixed.ipynb    # Fixed version — v1
├── nlp-twitter-sentiment-MAXACC.ipynb            # Final high-accuracy version
└── README.md                                     # GitHub documentation
```

Each notebook is a complete, standalone file. The progression from original to fixed to maximum-accuracy version is intentional — it shows the debugging and optimization process step by step rather than jumping straight to the final solution.

7. Key Takeaways

- **Debug before you optimize.** Fixing the bugs first before chasing accuracy was the right call. Spending time optimizing a pipeline that cannot run is wasted effort.
- **Data cleaning is 60% of the work.** The cleaning pipeline had a bigger impact on accuracy than switching models. What you feed the model matters more than which model you use.
- **Char n-grams are underrated.** Most tutorials stop at word n-grams. Character-level TF-IDF added measurable improvement for noisy Twitter text.
- **Ensembles almost always win.** Even when one model slightly outperforms the others individually, soft voting over three models gives more stable, higher accuracy results across different random seeds.
- **Simple features count.** Slang expansion, CAPS ratio, and punctuation counts are simple to implement and easy to explain. They worked.

8. How to Run

The easiest way to run this project is directly on Kaggle, where the dataset is already linked and all dependencies are pre-installed.

1. Open the notebook on Kaggle (recommended)
2. Add the dataset: jp797498e/twitter-entity-sentiment-analysis
3. Open nlp-twitter-sentiment-MAXACC.ipynb and run all cells top to bottom

No additional installs are required in the Kaggle environment. For local runs, install the standard data science stack: pip install scikit-learn nltk pandas numpy matplotlib seaborn scipy.

Built by Vinitha — BCA (New Horizon College, 8.8 CGPA) + BSc Data Science, IIT Madras

NLP | Machine Learning | Python | scikit-learn